The role of algorithms in computing

Part I Foundations

Scope

Specify legal inputs, acceptable outputs, and termination before comparing algorithms, representations, and hardware under one cost model. Duplicate-key sorting, paths and tours, and a numerical crossover distinguish general correctness, randomized guarantees, and finite testing.

Prerequisites

Basic functions, sets, program execution, and discrete input encodings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Distinguish a problem, its input domain, its output relation, and an algorithm specification.
- Describe algorithm quality through correctness, termination, and resource growth.
- Distinguish abstract efficiency from hardware-dependent constants.
- 1.1 Algorithms
- 1.2 Algorithms as a technology

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.1 Algorithms

A computational problem specifies a legal input domain D and a set $R(x)$ of acceptable outputs for each $x \in D$. The same keys may admit several sorted outputs, so a specification is generally a relation rather than a uniquely valued formula. An instance is one particular x. An algorithm is an unambiguous executable procedure that transforms inputs into outputs.

Total correctness of a deterministic algorithm A requires termination and $A(x) \in R(x)$ for **every** $x \in D$. Partial correctness requires only that a terminating execution produce a legal result. Termination needs a separate proof. The behavior outside the legal domain, such as rejection, an undefined result, or a separate return value, must also be specified.

A finite program can describe an infinite loop. Finitely many pseudocode lines do not prove finite execution. Tests on selected inputs do not establish the universal quantifier in the specification.

Sorting preserves order and every occurrence

Let $A = [(3, a), (1, b), (3, c)]$, with records represented by (key, identity). Keys have a total order, and the comparison cost is specified separately. A legal sorted output must have nondecreasing keys and be a permutation of the same input records, with neither losses nor copies. The output $[(1, b), (3, a), (3, c)]$ satisfies both conditions. The ordered output $[(1, b), (3, a)]$ loses record c and is invalid.

Stability additionally preserves input order among equal keys, requiring a before c. The output $[(1, b), (3, c), (3, a)]$ satisfies ordering and preservation but fails stability. An empty array returns an empty array and satisfies both basic conditions.

If keys include values without ordinary total-order semantics, define a comparison policy first. Incomparable values cannot silently count as ordered. Record identities make preservation and equal-key order directly observable.

A completely provable algorithm

For an array of length $n \geq 0$, find the smallest index i with $A[i] = v$, or return NIL if none exists. Indices start at 1. Logical `and` uses left-to-right short-circuit evaluation.

```
FIRST-MATCH(A, v)
    n = A.length
    i = 1
    while i <= n and A[i] != v
        i = i + 1
    if i <= n
        return i
    return NIL
```

Before each loop test, $A[1..i-1]$ contains no occurrence of v. The empty initial prefix satisfies this invariant. Each iteration confirms $A[i] \neq v$ before advancing, preserving it. At exit, if $i \leq n$, the current value equals v and all earlier values differ, so this is the smallest matching index. Otherwise $i = n + 1$, and no array element matches. The variant $n - i + 1$ is nonnegative inside the loop and decreases by 1 per iteration. At most n iterations occur, proving termination.

Success, failure, and empty input

For $A = [4, 9, 4]$, searching for 4 starts with $i = 1$. The bound $1 \leq 3$ holds and $A[1] = 4$, so the loop stops and returns 1. The second occurrence at index 3 does not change the smallest index.

Searching the same array for 7 gives the complete trace below.

Index before test	$i \leq n$	Key comparison	Next state
1	true	$4 \neq 7$	$i = 2$
2	true	$9 \neq 7$	$i = 3$
3	true	$4 \neq 7$	$i = 4$
4	false	do not read $A[4]$	return NIL

For $n = 0$, the first bound test is false and $A[1]$ is never read. Bounds and short-circuit semantics are part of implementable pseudocode.

Similar problems can have different legal answers

Same input, different output contracts



Same complete undirected graph. Crossing diagonals do not add a vertex. Only a tour must visit all vertices and return.

In this complete undirected graph, the four perimeter edges have weight 1 and both diagonals have weight 2. A shortest path from 0 to 2 costs 2, using either $0 \to 2$ or $0 \to 1 \to 2$; visiting 3 is unnecessary. TSP optimization requires visiting every vertex exactly once and returning to the start. The minimum tour $0 \to 1 \to 2 \to 3 \to 0$ costs 4.

Similar problems can have different legal answers (continued)

Fixing start 0 and identifying a tour with its reversal leaves three tours. Their sequences and costs are 01230 with cost 4, 01320 with cost 6, and 02130 with cost 6. Exhaustive enumeration proves optimality for **this four-vertex instance**. It is not a polynomial-time algorithm for general TSP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Representation determines feasibility and search space

LCS asks for a subsequence that preserves order but need not be contiguous. For $X = ABA$ and $Y = BAA$, both AA and BA are common subsequences of length 2. A length-3 subsequence would have to equal the whole string, but the strings differ, so the optimum is 2. A common substring additionally requires contiguous positions and defines a different problem.

The eight position subsets of X produce $\varepsilon, A, B, A, AB, AA, BA, ABA$. Repeated characters let different subsets produce the same string. Thus 2^n counts position choices, not necessarily distinct strings.

The dependency edges $a \to c, b \to c, c \to d$ require both a, b before c, and c before d. Orders a, b, c, d and b, a, c, d are valid; a, c, b, d is not. Adding $d \to b$ forces $b < c < d < b$, so no topological order exists. A graph representation makes such infeasibility explicit.

Quantifiers in hardness and approximation

A decision problem asks yes or no; an optimization problem asks for the best value or solution. TSP decision asks whether some tour costs at most B; optimization finds a minimum-cost tour. NP-complete usually describes the decision version; the optimization version is NP-hard. State which version is meant.

The reduction property of NP-completeness says that a polynomial-time algorithm for one NP-complete decision problem would yield polynomial-time algorithms for all NP problems. The label alone does not prove that a polynomial-time algorithm is impossible; that conclusion requires the appropriate complexity separation premise.

An approximation guarantee needs a feasible output and an explicit objective bound, such as $\mathrm{cost}(output) \leq \rho\, OPT$ for minimization. A plausible-looking answer does not establish this bound. Heuristics, exact exponential algorithms, restricted input classes, and approximation algorithms address different requirements, with distinct time and correctness assumptions.

State the source of randomness

Fix a legal input x and let r be the internal random bits. A Monte Carlo guarantee may state bounded running time and $\Pr_r[A(x;r) \text{ is correct}] \geq 2/3$. This is not a claim that random testing passes on two thirds of inputs. A guarantee for every x cannot be replaced by an unspecified average over inputs.

Three independent runs of the same binary decision followed by majority vote have error probability at most

$$\binom{3}{2}(1/3)^2(2/3) + (1/3)^3 = 7/27,$$

so success is at least $20/27$. Independence is essential. Sharing a random choice may make all three runs fail together. Search problems can have several distinct correct outputs, so literal string majority does not automatically apply.

Zero-error randomized algorithms need a separate termination and expected-time specification. Finite expected running time implies almost-sure termination, but need not give a uniform finite bound for every infinite random-bit sequence. These quantifiers differ from deterministic total correctness.

Data structures and parallel costs

An arbitrary array lookup may scan n elements. A sorted array permits binary search, but insertion may move $\Theta(n)$ elements. Hash-table expectations depend on the hash and load model. No representation unconditionally makes every operation constant-time.

Consider four independent unit tasks followed, after all four finish, by a chain of four unit tasks. Work is $W = 8$; span is $S = 5$. Completion times on 1, 2, and 4 processors are 8, 6, and 5. For every P,

$$T_P \geq \max(W/P, S), \qquad W/T_P \leq W/S = 1.6.$$

The ratio W/S bounds speedup. It does not mean that processors beyond 1.6 cannot help. Here $P = 2 > 1.6$, yet increasing to 4 reduces time from 6 to 5. Achievable times also depend on the DAG and the integer processor count.

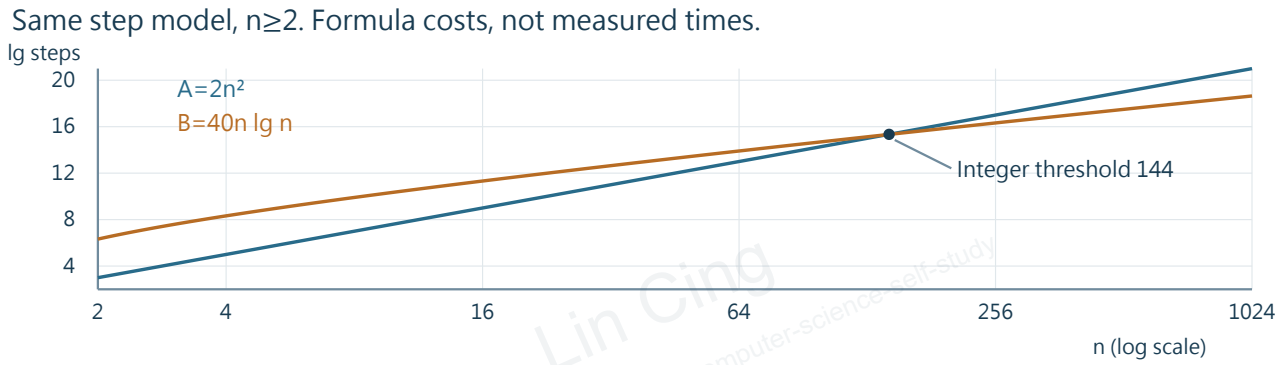
1.2 Algorithms as a technology

Algorithms and hardware jointly determine time. If the operation count is $I(n)$ and the effective processing rate is R, a simplified model gives $T(n) = I(n)/R$. Comparisons must use consistent input sizes, operation definitions, rates, and units. Memory, I/O, and synchronization can invalidate a single-rate model.

Let $I_A(n) = 2n^2$ and $I_B(n) = 40n \lg n$, for $n \geq 2$, using identical abstract steps and the same R. Algorithm A has a smaller constant and quadratic growth; B has $n \log n$ growth. These are explicit models, not fixed exact instruction counts for every insertion-sort or merge-sort implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check both sides of a crossover



The inequality $B < A$ is equivalent to $g(n) = n - 20 \lg n > 0$. Since $g'(x) = 1 - 20/(x \ln 2)$, the function is strictly increasing after $20/\ln 2$. We have $g(143) < 0$ and $g(144) > 0$, so B wins for every integer $n \geq 144$. The initial decrease followed by increase rules out a missed earlier interval for $n \geq 2$.

.

Lin Cing

Check both sides of a crossover (continued)

n	A steps	B steps	Fewer steps
16	512	2560	A
128	32768	35840	A
256	131072	81920	B
1024	2097152	409600	B

The figure uses logarithmic axes for this same model. A continuous intersection need not be the integer threshold. Strict inequality also requires checking possible equality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Convert hardware rates to the same time unit

Now give A rate $R_A = 10^9$ steps/s and B rate $R_B = 10^7$ steps/s. A has 100 times the raw throughput, while the operation formulas remain unchanged. At $n = 10^6$, A uses 2×10^{12} steps and takes 2000 s. B uses about 7.9726274×10^8 steps and takes 79.7263 s. Despite the slower processor, B finishes about 25.086 times as fast in this model.

At $n = 1000$, A takes .002 s and B about .0398631 s, so A wins. The time crossover now solves $n - 2000 \lg n > 0$; the equal-rate threshold 144 no longer applies.

Big-O denotes an upper-bound class and cannot directly compare implementations with unknown constants. The limit $n / \log n \to \infty$ supports eventual dominance of the lower growth rate when constants are fixed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Input length and available memory

A positive integer N has binary length $L = \lfloor \lg N \rfloor + 1$. A loop with N iterations is $\Theta(N)$ in the numerical value but can be $\Theta(2^L)$ in input bits. It is not generally linear in encoded input size. An array of n keys of w bits occupies $\Theta(nw)$ bits. Whether a comparison costs $O(1)$ depends on the word and cost model.

If memory holds only M records, an input with $n > M$ requires a different model. External I/O and buffering may dominate. Parallelism, caches, and networks are not automatically negligible constants; choose the input scale and cost boundary first.

Algorithms appear in applications, compilers, runtimes, operating systems, networks, and hardware. The transferable ability is to formulate contracts, choose representations, and prove correctness and cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study